

Day 1

- The migration files were created.
- A total of seven database tables were created successfully by running the `php artisan migrate` command.

Days 2–3

- The models and the relationships between them were created.
- The `Artist`, `Album`, `Genre`, `Song`, `Order`, and `OrderItem` models were created. The `User` model was already provided by Laravel.
- The `Artist::albums()` and `Album::artist()` relationships were defined.
- The `Album::genre()` and `Genre::albums()` relationships were created. An error related to the direction of the relationship was identified and corrected.
- The `Album::songs()` and `Song::album()` relationships were defined. An error that caused the model to reference itself was fixed.
- The `User::orders()` and `Order::user()` relationships were created. A camelCase naming error was also corrected.
- It was determined that there should not be a direct relationship between the `Order` and `Album` models. Therefore, the relationships were established through the `OrderItem` model:
 - `Order::orderItems()`
 - `Album::orderItems()`
 - `OrderItem::order()`
 - `OrderItem::album()`
- The model relationships were tested and verified using Laravel Tinker.

Days 4–5

- The factory and seeder sections were completed.
- Six factories were created for the `Artist`, `Genre`, `Album`, `Song`, `Order`, and `OrderItem` models.
- Faker method errors, such as the incorrect use of `word` and `words`, as well as `randomDigit` and `randomFloat`, were identified and corrected.
- The `HasFactory` trait was added to all models.
- A total of seven seeders were created.
- Fixed genre names were manually defined in `GenreSeeder` to avoid possible problems caused by the `unique()` method.

- All seeders were connected to `DatabaseSeeder` in the correct execution order by using `$this->call([...])`.
- The `php artisan migrate:fresh --seed` command was executed to test the complete database creation and data seeding process.

Days 6–7

- The routes, controllers, and views for the user-facing side of the application were completed.
- The `/albums` page was created.
- Because of a problem with the Herd domain, the application was temporarily run by using the `php artisan serve` command.
- The `/artists` page was created. An error involving singular and plural route naming was corrected.
- The `/genres` page was created.
- The `/genres/{id}/albums` route was created to display the albums belonging to a selected music genre.
- The `/albums/{id}` detail page was created, and the album was retrieved by using the `find($id)` method.
- The `/artists/{id}` detail page was created, and the albums belonging to the selected artist were displayed on the page.

Days 8–10

- All routes in the project were reviewed and compared with the project PDF document.
- A song list was added to the `/albums/{id}` album detail page.
- The `/orders` route was moved to `/admin/orders` to make it suitable for the administration panel.
- The `/profile/orders` page was created to display a user's orders. Since a real authentication system had not yet been implemented, `User::find(1)` was used as a temporary user simulation.
- An independent `SongController` and the `/songs` route were created.
- Development of the administration panel was started.
- It was decided that each model would have a separate administration controller.
- An `AdminController` was created for the administration dashboard.
- Six administration controllers were created for the `Artist`, `Album`, `Song`, `Genre`, `User`, and `Order` models.
- An `index()` method was initially added to each administration controller.
- `abort(404)` checks were added to prevent errors when queries made with the `find()` method returned no result.

- A `show()` method, detail route, and related view were added to every administration controller.
- A pool of 65 fictional artist names was added to `ArtistFactory`.
- `SongSeeder` and `AlbumSeeder` were rewritten to create matching data based on the Rock, Jazz, Hip Hop, Classical, and Electronic music genres.
- Order items were added to the `admin/orders/show` and `profile/orders` pages. Orders and order items were displayed by using nested `@foreach` structures.
- User order statistics were added to the `/profile/orders` page.
- Model and record statistics were added to the `/admin` dashboard by using the `Model::count()` method.

Days 11–12

- The `layouts/app.blade.php` layout was created for the user interface, and `layouts/admin.blade.php` was created for the administration panel.
- A color palette was defined with CSS variables inside the `:root` selector.
- The `public/css/style.css` file was created for the general design rules.
- A flip-card design using the 3D `rotateY` property was tested for album cards. It was later replaced with a simpler card design because it was not considered suitable for the application.
- Hover effects, including upward movement and shadows, were added to the header and navigation elements.
- The width and centering settings of the `main` section were configured.
- The album detail page was designed with a hero section and a song list.
- The artist listing and artist detail pages were designed.
- The genre listing and genre album pages were designed.
- The user profile page interface was created.
- A sidebar and an `admin-table` structure were added to the administration panel.
- The listing pages of all six models in the administration panel were converted into table layouts.
- `HomeController` and `home.blade.php` were created for the home page.
- Realistic artist, album, and song names were matched with music genres by using `whereIn` queries.
- The idea of adding YouTube links to songs was tested. However, the feature could not be fully completed because the links did not open the videos directly.
- The Tabler Icons webfont library was added to the project through a CDN.
- Artist biographies were updated by using more realistic template sentences.
- A song search feature was added to the `/songs` page.
- An album search feature was added to the `/albums` page.

- A general search box was added to the home page.
- The missing “Songs” and “My Orders” links were added to the navigation menu.

Day 13

- A general review of the project was performed.
- It was discovered that the `/admin/orders` route was loading the wrong view file, and the problem was corrected.
- An HTML structure error caused by nested `<nav>` elements was fixed.
- Unused files, including `orders/index.blade.php` and `welcome.blade.php`, were removed.
- An inconsistency in the use of the `country` field across the project was identified and corrected.

General Evaluation

During the development of the project, artificial intelligence tools were used to help plan the implementation steps and understand the errors encountered. Instead of directly copying the generated code, the code was reviewed line by line, manually written, and adapted to the structure of the project.

Throughout this process, practical experience was gained in model relationships, routing, controller operations, factories, seeders, Blade templates, administration panels, and user interface development. However, the project does not yet include a real user registration and login system, authorization, a complete purchasing workflow, or fully functional external media links. In its current form, the project has been completed as a basic virtual record store prototype.